



ELSEVIER

Int. J. Production Economics 43 (1996) 79–89

international journal of
**production
economics**

A tabu search heuristic for the multiprocessor scheduling problem with sequence dependent setup times

Paulo M. França^{a,*}, Michel Gendreau^b, Gilbert Laporte^b, Felipe M. Müller^c

^a Faculdade de Engenharia Elétrica, Universidade Estadual de Campinas, C. P. 6101, 13081-970 Campinas SP, Brazil

^b Centre de recherche sur les transports, Université de Montréal, C.P. 6128, succursale A, Montréal, Québec, Canada H3C 3J7

^c Departamento de Eletrônica e Computação, Universidade Federal de Santa Maria 97119-900 Santa Maria RS, Brazil

Received 9 February 1995; accepted for publication in revised form 13 November 1995

Abstract

This article deals with the problem of scheduling n jobs on m identical parallel processors with the objective of minimizing the total execution time (makespan). A new three-phase heuristic is proposed for solving this problem. An initial phase constructs a starting solution which is improved, in a second phase, by means of a tabu search method. A final phase follows attempting a further improvement in the current solution. Numerical tests in a series of randomly generated problems indicate that the proposed method outperforms a previous heuristic. Comparisons with an exact procedure attest that our method produces good-quality solutions in reasonable running times.

Keywords: Multiprocessor scheduling; Local search; Tabu search; Heuristics

1. Introduction

This paper addresses the problem of scheduling a given set of jobs to m identical parallel machines under the assumption that machines require a setup between jobs and also that the setup depends on the order of job processing. The objective is to minimize total completion time (makespan).

This problem arises frequently in production environments where the setup times (or costs) are significant. For instance, in a textile facility orders corresponding to a specific fabric type can be assigned to a set of available looms. The loom must

be equipped with a specific warp chain. The amount of time required to replace a warp chain depends on the fabric type previously processed on the loom. During the replacement operation the looms stay idle. Obviously, it is desirable to perform changeovers aiming to minimize the amount of time the looms remain idle.

In batch chemical production the time necessary for cleaning a reactor before a new product is processed depends on what was produced last in the reactor. A similar situation occurs in the plastic industry with the production of items of different colors. A set of orders constituted by items of different colors has to be assigned to a set of extrusion machines. When the color of an item previously processed in the machine is not the same of the one presently been processed, an amount of plastic is

* Corresponding author. Tel.: 55 192 397 302; fax: 55 192 391 395; e-mail: franca@densis.fee.unicamp.br

wasted until the standard color is reached. The sequence of items assigned to each machine will affect the total amount of wastage. Other similar examples can be found in the glass and paper industries.

In the scheduling literature these problems are known as sequence-dependent setup time (or cost) problems. A major classification divides the problems into single and parallel processor problems. In both cases other scheduling features may arise, such as due dates [1] or job preemption and job splitting [2]. When demand and production rates are considered, the model is known as the cyclic lot scheduling problem with sequence-dependent setups – SDLPS [3]. Concerning the objective functions adopted, problems with due dates generally attempt to minimize early/tardy completion costs [4, 5] or maximum tardiness [2]. In the absence of due dates the goal is the minimization of makespan [6] or flow time [7].

In this paper, attention is focused on the problem of scheduling identical parallel machines with sequence-dependent setup times in a make-to-stock environment. In such a situation, satisfying due dates is a not a main concern. Instead, production systems are primarily oriented towards minimizing lost production capacity due to changeovers. Restricting ourselves to the scheduling literature that tackles make-to-stock sequence-dependent problems, we observe that few algorithms have been proposed. An article by Dearing and Anderson [8] reports results obtained from an integer linear programming model applied to loom assignment in a textile weaving operation. They solved the problem using linear programming and rounding. Sumichrast and Baker [9] propose a heuristic method based on the solution of a series of 0–1 integer subproblems that improves on the results of Dearing and Anderson.

The models formulated by Sumichrast and Baker and Dearing and Anderson differ from our model because they consider that a job, characterized as a demand expressed in machine-days, can be split among several machines. In our model, job splitting is not allowed for quality control reasons. This makes the problem combinatorially much more complex. They also consider a different objective function which is to minimize the total setup cost.

Our model can be viewed as an extension to parallel machines of the well-known single-machine sequence-dependent setup time problem which Baker [6] has shown to be equivalent to the Traveling Salesman Problem (TSP). Another way to see our model is to consider it as an extension of the widely studied scheduling problem known as $P||C_{max}$, where the sequence-dependent constraint is dropped [10]. In the $P||C_{max}$ model a set of jobs has to be assigned to a set of identical parallel machines aiming to minimize the makespan. Job preemption and job splitting are not allowed.

Following the three-field classification introduced in [11], our problem can be categorized as $P|sds|C_{max}$. To the authors' knowledge only one paper [12] has reported computational results for this problem. Two other papers deal with similar problems. No exact algorithm has ever been proposed. In their article, Frederickson et al. provide two approximate algorithms for the $P|sds|C_{max}$. Both are derived from an equivalence between the $P|sds|C_{max}$ and the TSP, and make use of known TSP-based heuristic procedures. In another paper, Parker et al. [13] solve a similar problem where the objective is to achieve minimum total setup cost, instead of the makespan minimization. They impose a workload restriction that limits the capacity of each processor. The solution approach uses an equivalence with a vehicle routing problem that is solved by a heuristic procedure based on the classical Clarke and Wright [14] algorithm. Focusing on scheduling problems found in the textile industry, Guinet [15] proposes several algorithms. One of these problems is quite similar to the $P|sds|C_{max}$, except its objective function is to minimize the sum of the completion time of all jobs, instead of the makespan. It is not difficult to see that these objective functions can lead to different final solutions.

This paper considers the problem of non-preemptively scheduling n jobs, $J_1, J_2, \dots, J_n$, to m identical parallel processors, $M_1, M_2, \dots, M_m$, to minimize the completion time for the entire set of jobs. Each job J_j is assigned an integer and positive processing time p_j and an integer and positive setup time s_{ij} , if job J_j is executed immediately after job J_i . The setup times may be considered asymmetric, that is, $s_{ij} \neq s_{ji}$. The $P|sds|C_{max}$ is a difficult combinatorial problem proved to be NP-hard in a strong

sense because it is equivalent to the TSP when the number of processors equals one [6].

We propose a new heuristic procedure composed of three phases:

Phase 1: An initial solution is constructed by assigning all jobs to the m processors.

Phase 2: A local search heuristic controlled by a Tabu Search scheme is used to achieve improved solutions in terms of makespan moving jobs between different processors.

Phase 3: This is a final phase where one tries to improve the sequence obtained in the busiest processor to achieve a better makespan.

The next section explains the heuristic procedures for selecting the jobs that will be removed and assigned to processors. Note that during Phase I, jobs are assigned to processors and during Phase II there is a job reassignment process where a job is removed from a processor and assigned to another one. In Section 3 a detailed description of the three phases will be presented. In Section 4, the proposed heuristic is then compared with two other methods. The first one is the well-known nearest-neighbor heuristic of Frederickson et al. [12]. The absolute performance of the method is also evaluated by comparing it with an exact algorithm that uses a dichotomous search to achieve optimal solutions. Although the exact method constitutes by itself an original contribution, we choose not to present it in detail to save space. In this article it is used exclusively for benchmarking purposes.

2. Insertion and removal procedures

This section describes how jobs are inserted and removed from the processors. The procedures used here are adaptations for the scheduling environment of two heuristics, GENI and US, originally developed by Gendreau et al. [16] for the symmetric TSP. GENI is more powerful than standard insertion methods because it permits that every job may only be included in a processor containing one of its nearest neighbors, and every insertion is executed simultaneously with a local reoptimization of the current processor. Originally, US was conceived as a post-optimization phase where jobs are

successively removed from a processor, using the reverse of the GENI actions, and then reinserted in the same processor using GENI.

Defining the data structure used for implementing the asymmetric adapted versions of GENI and US, consider the arrangement depicted in Fig. 1.

Suppose that processor M_1 contains five jobs. This arrangement can be represented by a linked list where the processor is the head of the list and the last job points to the processor. Since the nature of setup times is sequence dependent and asymmetrical, this representation is very appropriate because it facilitates the inversion of part of the sequence of jobs and even the whole sequence as it will be required in the insertion procedure. Consider also that the matrix $[s_{ij}]$ representing the setup times should include an additional column where the element s_{0i} represents the setup time of job i , the first job in the sequence. Assume that $s_{j0} = 0, j = 1, \dots, n$.

The main feature of the insertion procedure is that when a job J_x is inserted in a processor, it is not necessarily placed between two consecutive jobs. However, after the insertion, these jobs become adjacent to J_x . For a given orientation of the sequence, consider J_k a job in the subsequence from J_j to J_i and J_l a job in the subsequence from J_i to J_j . Consider also the legend for the figures as given in Fig. 2.

The adaptation of GENI insertion procedure suggests two possible types of insertion.

Type I: Assume that $J_k \neq J_i$ and $J_k \neq J_j$. Inserting J_x in, say, processor M_m , results in the deletion of links (J_i, J_{i+1}) , (J_j, J_{j+1}) and (J_k, J_{k+1}) and their replacement by links (J_i, J_x) , (J_x, J_j) , (J_{i+1}, J_k) and (J_{j+1}, J_{k+1}) . Fig. 3 shows the type I insertion. Note that the subsequences $(J_{i+1}, \dots, J_j)$ and $(J_{j+1}, \dots, J_k)$ should appear inverted.

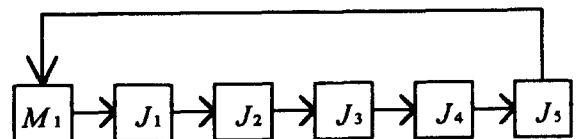


Fig. 1. Data structure representing five jobs on processor M_1 .

Type II: Assuming that $J_k \neq J_j$, $J_k \neq J_{j+1}$, $J_l \neq J_i$ and $J_l \neq J_{i+1}$, the insertion of J_x in processor M_m results in the deletion of links (J_i, J_{i+1}) , (J_{l-1}, J_l) , (J_j, J_{j+1}) and (J_{k-1}, J_k) that should be replaced by links (J_i, J_x) , (J_x, J_j) , (J_l, J_{l+1}) , (J_{k-1}, J_{l-1}) and (J_{i+1}, J_k) . As a consequence, the subsequences $(J_{i+1}, \dots, J_{l-1})$ and $(J_l, \dots, J_j)$ are inverted, as can be shown in Fig. 4.

To cope with the asymmetric aspect inherent to the $P|sds|C_{max}$, two other types of insertion can be defined.

Type III: This type of insertion can be seen as the inverse of Type I insertion. When job J_x is inserted between J_i and J_j , the sequence of jobs is rearranged in such way that almost the whole sequence is inverted (see Fig. 5). The aim is to explore other regions of the search space, hoping that valleys of new promising solutions will be found. As in Type I insertion, we assume that $J_k \neq J_i$, $J_k \neq J_j$. The insertion of J_x implies the deletion of links

(J_{i-1}, J_i) , (J_{j-1}, J_j) and (J_{k-1}, J_k) that should be replaced by links (J_i, J_x) , (J_x, J_j) , (J_k, J_{j-1}) and (J_{k-1}, J_{i-1}) . This cause the inversion of the sequences $(J_i, \dots, J_{j-1})$ and $(J_k, \dots, J_{i-1})$.

Type IV: Analogously, Type IV insertion is the reverse of Type II (Fig. 6). Again it is supposed that $J_k \neq J_j$, $J_k \neq J_{j+1}$, $J_l \neq J_i$ and $J_l \neq J_{i+1}$. The links deleted are (J_{i-1}, J_i) , (J_l, J_{l+1}) , (J_{j-1}, J_j) and (J_k, J_{k+1}) , and the links added are (J_i, J_x) , (J_x, J_j) , (J_{i-1}, J_l) , (J_{l+1}, J_{k+1}) and (J_k, J_{j-1}) . The subsequences that are inverted as a result are $(J_i, \dots, J_l)$ and $(J_{l+1}, \dots, J_{i-1})$.

Since the potential number of choices for J_i , J_j , J_k and J_l could reach a large number, we limit the search by introducing the concept of *local neighborhood*. Consider q as an input parameter. The local neighbors of a job J_j , relative to a processor, are the closest q successors and the closest q predecessors among the jobs already assigned to this processor. The closest successor of job J_j means the job J_k with the least s_{jk} , and the closest predecessor is job J_i with the least s_{ij} . The selection of the best place to insert a job in a processor is now constrained to the local neighborhood of each job involved in the alternative under consideration. For this reason, local neighborhood is defined for each processor, i.e., each job has m local neighborhoods. The example below

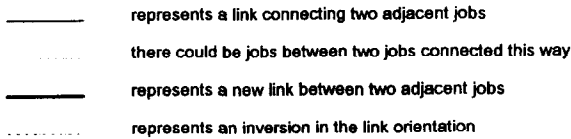


Fig. 2. Legend for the figures.

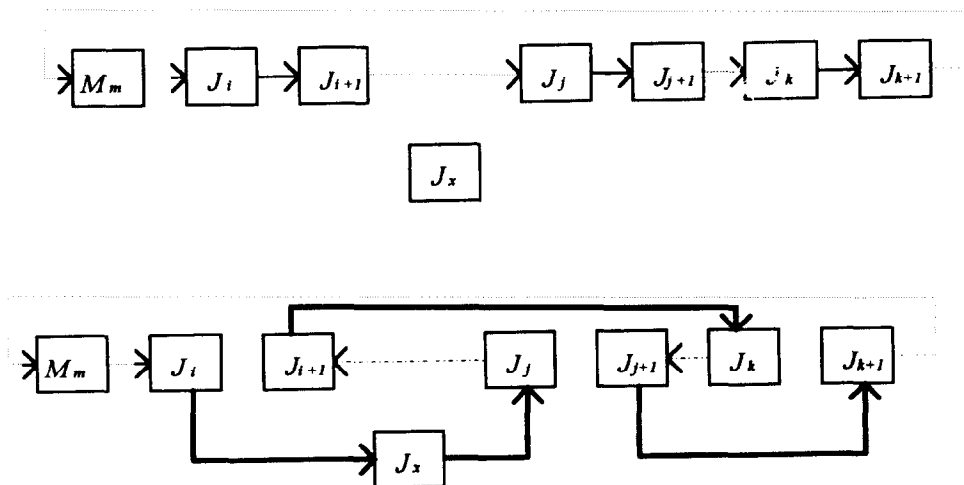


Fig. 3. Type I insertion.

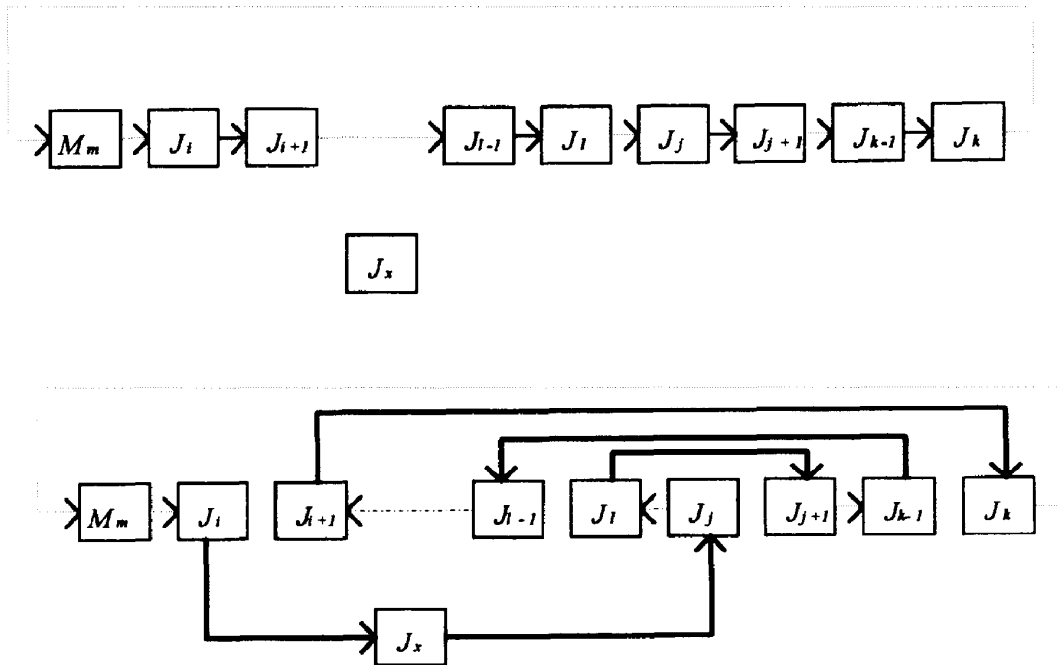


Fig. 4. Type II insertion.

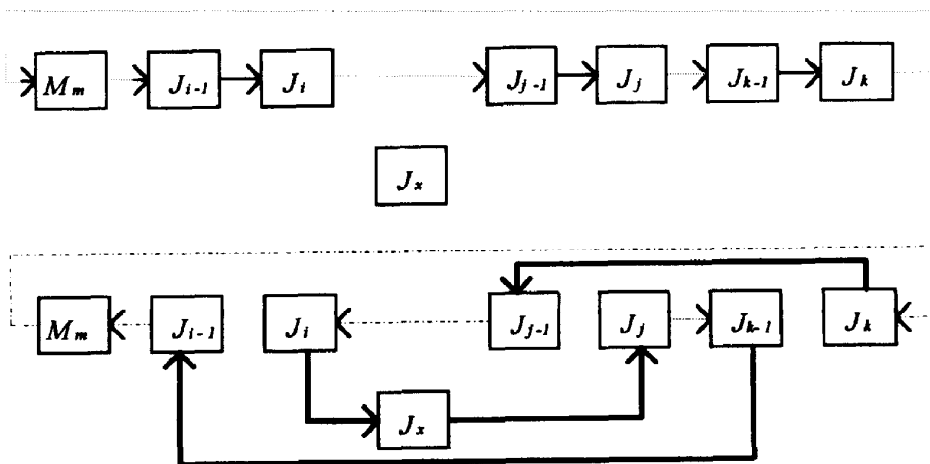


Fig. 5. Type III insertion.

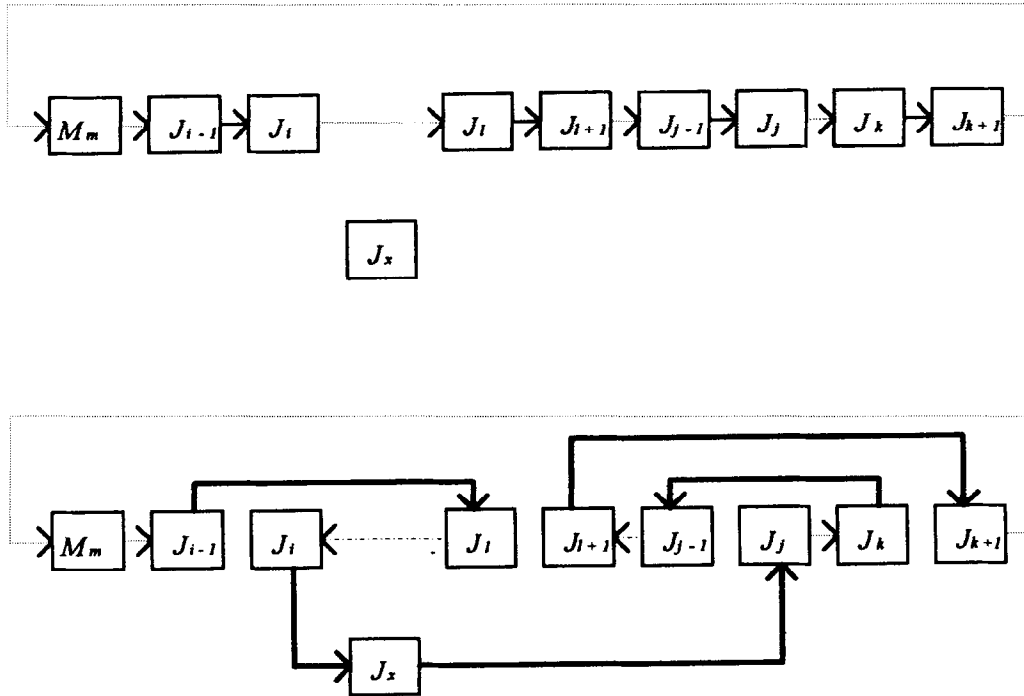


Fig. 6. Type IV insertion.

illustrates this concept. Consider the following matrix $[s_{ij}]$.

s_{ij}	0	1	2	3	4	5	6	7
0	—	32	30	54	9	42	38	42
1	0	—	63	47	25	45	7	50
2	0	21	—	76	38	60	69	56
3	0	33	26	—	54	16	44	23
4	0	58	61	37	—	45	32	46
5	0	49	49	25	13	—	45	8
6	0	23	29	18	36	27	—	51
7	0	74	82	61	25	38	54	—

Suppose that there are two processors and the jobs currently assigned to them are:

M_1 : 0 – 2 – 7 – 5, M_2 : 0 – 3 – 1 – 4 – 6.

The local neighborhood structure for job J_1 and $q = 2$ is then

M_1
 successors = {0, 5}
 predecessors = {2, 0}

M_2
 successors = {0, 6}
 predecessors = {6, 0}

Phase II of the method also uses the concept of *global neighborhood*. Global neighbors of a job J_j are its p closest successors and its p closest predecessors among all jobs, except J_j itself. Global neighborhood is useful to constrain the search space when the procedure is selecting a processor to insert job J_j . The processors chosen as candidates are only the ones that possess a minimum number of J_j 's *global neighbors*. Using the same example and supposing $p = 3$, the J_1 global neighbors are:

successors: {0 – 6 – 4}
 predecessors: {2 – 6 – 0}

The removal process is the reversal of GENI insertion process. To each type of insertion there is a correspondent type of removal. Fig. 7 shows the Type I removal that incorporates the local

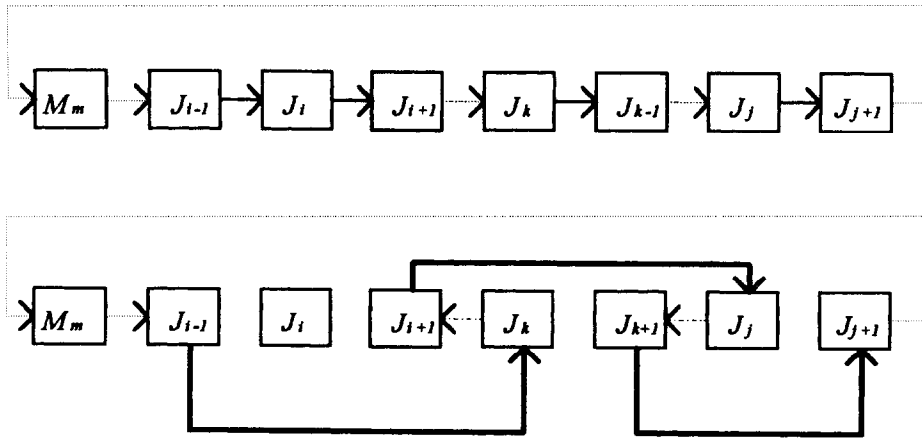


Fig. 7. Type I removal.

neighborhood structure defined above. The other three types of removal can be derived in a straightforward manner and are omitted for simplicity.

Type I removal: Consider J_j belonging to the local neighborhood of J_{i+1} and J_k belonging to the local neighborhood of J_{i-1} , with J_k being part of the sequence $(J_{i+1}, \dots, J_{j-1})$. The removal of job J_i consists in the deletion of links (J_{i-1}, J_i) , (J_i, J_{i+1}) , (J_k, J_{k+1}) and (J_j, J_{j+1}) and in the addition of links (J_{i-1}, J_k) , (J_{i+1}, J_j) and (J_{k+1}, J_{j+1}) . The sequences $(J_{i+1}, \dots, J_k)$ and $(J_{k+1}, \dots, J_j)$ become inverted.

3. A Tabu Search Algorithm

Tabu Search (TS) is a global optimization meta-heuristic, first proposed by Glover [17, 18], that explores the solution space by repeatedly moving from a solution to the best of its neighbors. A key difference from the so-called hill-climbing algorithms resides in the fact that TS is not trapped to local minima. The search process is provided with a mechanism that allows the objective function to deteriorate, and in a controlled way it is able to escape from local optima. Starting from an initial solution, an admissible move leads to the next solution selected as the one with the minimum cost. If this solution is a local minimum, this means accepting a nonimproving perturbation. Since the

search always chooses the best new movement it might well fall back into the local minimum from which it previously emerged. To prevent cycling, the reverse of a movement that has been recently performed is forbidden (tabu) and inserted in a constantly updated *tabu list*. In recent years, a number of TS heuristics have been successfully applied to a variety of combinatorial problems [18].

TS methods are open-ended and the key to efficient implementation is that they should be tailored to the characteristics of the problem at hand. Some of the key features that are responsible for a good TS implementation are:

- The neighborhood structure;
- The initial solution;
- The tabu tenure, i.e., for how long a tabu move will be enforced;
- The stopping criterion.

In our TS implementation for solving the $P|sds|C_{max}$, a neighbor solution is obtained by removing a job from the busiest processor and inserting it in another processor. The whole neighborhood is then constructed using the concepts of global and local neighborhood previously defined. It is obtained by taking every job J_j belonging to the busiest processor and then forming a new solution by inserting this job in one of the processors of the set $M(J_j)$. The set $M(J_j)$ contains the processors that are candidates to receive job J_j and is defined

as $M(J_j) = \{M_i / M_i \neq \bar{M}_i \text{ and } M_i \text{ includes among its jobs at least one } J_j\text{'s global predecessor or successor}\}$, where $\bar{M}_i$ is the busiest processor.

Let $C(M_i)$ be the finishing time to execute all jobs assigned to processor M_i . In the description of the algorithm below consider that when we refer to insertion or removal of jobs we mean the movement that causes the least value of $C(M_i)$ considering every insertion/removal type described in the preceding section. In other words, we are referring to the *best* insertion or removal action.

A move can be then characterized by a vector $(J_j, \bar{M}_i, M_i^*)$ associated with the job J_j which is taken out from the busiest processor and transferred to the processor M_i^* which has been selected from the candidate set $M(J_j)$. After a movement is performed, the reverse action is prohibited for the next θ iterations. The parameter θ plays an important role in the performance of most of the TS implementations. If it is too small, the probability of cycling increases. On the other hand, if it is too large, there is a possibility that all but a few of the moves available in the neighborhood of the current solution are tabu and the algorithm may perform poorly because the search space is too restricted. The appropriate choice for θ and other parameters is discussed at the end of the section. The stopping criterion was chosen as the number of iterations, T , during which there may be no improvement in the best solution found so far (incumbent solution).

The initial solution is obtained by successively assigning each job to the processor that yields the least increase in the current makespan. The process also takes into consideration all types of insertion previously defined.

PHASE I: Initial Assignment

Step 1: Find the p global predecessors and the p global successors of each job J_j .

Step 2: Choose at random m jobs and successively assign one to each processor. For each job and each processor, construct the list of local neighbors.

Step 3: Insert an unassigned job to the processor that yields the least increase in the current finishing time. After every insertion, update the list of local neighbors. When all jobs have been assigned, stop.

Store the initial solution as the incumbent solution and go to Phase II.

PHASE II: Tabu Search

Suppose that $\bar{M}_i$ is the busiest processor and consider

$C(M_i, J_j)$ = finishing time associated with processor M_i if job J_j is inserted in it.

$C(\bar{M}_i, J_j)$ = finishing time associated with processor $\bar{M}_i$ if job J_j is removed from it.

Step 1: Set the iteration counter $t = 0$.

Step 2: For each job J_j belonging to the busiest processor, compute:

- the set of processors that are candidates to receive job J_j , defined as: $M(J_j) = \{M_i / M_i \neq \bar{M}_i \text{ and } M_i \text{ includes among its jobs at least one } J_j\text{'s global predecessor or successor}\}$;
- the minimum finishing time caused by the insertion of job J_j in one of the processors of set $M(J_j)$, defined as

$$C(M_i^*, J_j) = \min\{C(M_i, J_j) / M_i \in M(J_j)\}.$$

- the maximum finishing time if job J_j is moved from $\bar{M}_i$ to M_i^* , defined as

$$T_j = \max\{C(\bar{M}_i, J_j), C(M_i^*, J_j)\}.$$

Step 3: Select job J_j^* to be removed from $\bar{M}_i$ as $J_j^* = \text{argmin}\{T_j\}$ unless the move is tabu, insert it in M_i^* and declare the reverse move tabu for θ iterations. If the selected move is tabu but it yields a better makespan, override the tabu status and perform the move.

Step 4: Update the lists of q local predecessors and successors affected by the changes in processors $\bar{M}_i$ and M_i^* . Identify the busiest processor and update the incumbent solution, if necessary.

Step 5: Set $t = t + 1$. If T , the maximum number of iterations without improvement has not been reached, go to *Step 2*. Otherwise, go to Phase III.

PHASE III: Final Adjustment

Step 1: Consider $\bar{M}_i$ the busiest processor in the incumbent solution and $C(\bar{M}_i)$ its finishing time. For each job assigned to $\bar{M}_i$ perform a removal and an insertion in the same processor generating a new finishing time $C'(\bar{M}_i)$. Define

$C^*(\bar{M}_i) = \min_{j \in \bar{M}_i} \{C'(j)\}$ as the minimum finishing time found after all removals and insertions.

Step 2: If $C^*(\bar{M}_i) \geq C(\bar{M}_i)$, the minimum finishing time does not alter, stop.

If $C^*(\bar{M}_i) < C(\bar{M}_i)$, but $\bar{M}_i$ still is the busiest processor, update the incumbent solution and stop. Otherwise, update the incumbent solution and go to *Step 1*.

Some comments on the values adopted for the parameters follow. The values of p and q are taken as

$$p = \lceil \alpha \sqrt{n} \rceil \quad \text{and} \quad q = \max \left[3, \lceil \alpha \sqrt{\frac{n}{m}} \rceil \right],$$

where α is an input parameter. With these rules the size of global and local neighborhoods increases with the problem dimension but does not become unmanageable as n increases. Tests with α ranging in the interval $[0.1, 2.0]$ have shown that $\alpha = 0.6$ is a good trade-off between good quality solutions and computational time. As in [19], the number of iterations θ for which a move is declared tabu is not a constant, but a random variable uniformly distributed in the interval $[\underline{\theta}, \bar{\theta}]$. We have tested ten choices of these parameters in the range $[3, 20]$, and concluded that $\underline{\theta} = 3$ and $\bar{\theta} = 7$ was the best choice. Note that in order to reduce the amount of book-keeping required, no tabu list is actually maintained in our implementation. Instead, we simply use a “tabu tag” θ to indicate for how many iterations a particular move is declared tabu. The number T of iterations without improvements was set to $10n$.

4. Computational results

The proposed Tabu Search algorithm was compared with the heuristic of Frederickson et al. [12] and with an exact procedure. We have implemented the nearest-neighbor heuristic suggested in the paper of Frederickson et al. and adapted it to solve the $P|sds|C_{max}$.

Optimal solutions can be obtained for medium-sized instances making use of an algorithm originally developed for solving a closely related problem, the Distance Constrained Vehicle Routing Problem (DVRP). The similarity between vehicle routing problems (VRP) and parallel processor

problems is well known. In VRP a set of n demand points are to be serviced by at most a set of m vehicles. The jobs and processors correspond to demand points and routes, respectively. The paper of Parker et al. [13] makes use of this parallelism to solve a problem similar to the $P|sds|C_{max}$.

In the DVRP every route is constrained not to exceed a preset constant D . We propose an optimal algorithm that uses a dichotomous search scheme where a series of DVRP is solved. At each iteration the value of D is adjusted to constrain the search. Any exact algorithm for the DVRP can be used. We have chosen to implement the Laporte et al. [20] algorithm which was already available.

Two different versions of the Tabu Search heuristic were implemented. *Fast Tabu* uses a fixed value of 0.6 for α and *Long Tabu* tries to achieve better solutions allowing α to assume four different values: 0.6, 0.8, 1.5 and 2.0. The best solution is chosen.

We compare the results over different combinations of the number of jobs and processors available. For each combination of $n = 20, 30, 40, 50$ and $m = 2, 3, 4, 5$, ten different randomly generated problems were solved. The processing times and the setup times were generated from a uniform distribution in $[1, 10]$ and $[0, 10]$, respectively. During the execution of the optimal algorithm, if the running time to solve a DVRP exceeds 1000 s then the instance is discarded. Table 1 reports average results over the number of successful instances, as well as the best and worst values achieved. When not all instances can be solved, the number of successful cases is indicated by the number in parenthesis. All problems were solved on a Sun SPARC2 workstation. Table 2 shows the average running times and the best and worst performance for Fast Tabu and Long Tabu implementations.

The terminology used in Tables 1 and 2 is as under:

n	number of jobs
m	number of processors
Z_{FHK}	Frederickson, Hecht and Kim's heuristic solution value
Z^*	optimal solution value
Z_{FT}	Fast Tabu solution value
Z_{LT}	Long Tabu solution value
FHK	($= Z_{\text{FHK}}/Z^*$)
FT	($= Z_{\text{FT}}/Z^*$)

FT_{best} (=best result for Fast Tabu); FT_{worst} = worst result for Fast Tabu
 LT (= Z_{LT}/Z^*)
 LT_{best} (= best result for Long Tabu); LT_{worst} = worst result for Long Tabu
 t corresponding CPU time in seconds

Table 1
Computational results

n	m	FHK	FT	LT	FT _{best}	FT _{worst}	LT _{best}	LT _{worst}
20	2	1.25	1.03	1.02	0.0	1.05	0.0	1.04
	3	1.29	1.04	1.03	1.03	1.06	1.03	1.05
	4(6)	1.33	1.04	1.03	0.0	1.07	0.0	1.04
	5(6)	1.39	1.07	1.04	1.04	1.10	0.0	1.05
30	2	1.22	1.05	1.03	1.03	1.07	1.01	1.04
	3(7)	1.24	1.05	1.03	1.03	1.10	1.02	1.04
	4(4)	1.31	1.07	1.05	1.05	1.08	1.05	1.05
	5(2)	1.34	1.09	1.06	1.09	1.10	1.06	1.07
40	2	1.20	1.02	1.02	1.02	1.04	1.01	1.04
	3(6)	1.23	1.05	1.03	1.04	1.06	1.01	1.05
	4(3)	1.20	1.06	1.04	1.05	1.06	1.03	1.04
	5(3)	1.31	1.05	1.04	1.03	1.06	1.03	1.06
50	2	1.17	1.03	1.02	1.02	1.05	1.01	1.03
	3(6)	1.18	1.04	1.03	1.03	1.05	1.03	1.04
	4(5)	1.26	1.05	1.03	1.05	1.07	1.02	1.05
	5(3)	1.24	1.06	1.04	1.05	1.06	1.03	1.04

Table 2
Running times

n	m	t_{FHK}	t_{FT}	t_{LT}	t_{FTbest}	$t_{FTworst}$	t_{LTbest}	$t_{LTworst}$
20	2	0.002	8.6	170.6	6.1	10.3	137.6	231.6
	3	0.003	5.7	83.0	4.0	9.1	60.8	109.5
	4(6)	0.003	4.2	35.4	3.2	5.0	29.3	43.8
	5(6)	0.003	3.2	28.3	2.6	5.4	21.6	32.4
30	2	0.003	19.1	1263.2	15.3	30.7	894.0	1871.1
	3(7)	0.012	15.8	477.4	12.4	22.3	416.6	622.1
	4(4)	0.004	10.5	242.0	9.5	12.3	197.0	274.7
	5(2)	0.008	9.1	147.6	8.0	10.2	137.7	157.5
40	2	0.002	47.7	4052.4	33.9	68.3	3061.5	5896.8
	3(6)	0.011	36.7	1699.4	23.8	60.9	1375.6	2204.5
	4(3)	0.011	28.4	835.8	21.5	32.0	690.3	1003.9
	5(3)	0.017	28.3	438.6	20.2	33.7	387.9	518.3
50	2	0.009	94.4	9377.9	62.3	162.3	7525.3	12282.8
	3(6)	0.006	62.9	4535.6	44.8	101.1	3513.7	5692.9
	4(5)	0.017	52.6	2632.4	34.8	72.6	2283.1	3220.1
	5(3)	0.028	35.0	1482.4	27.0	46.8	1150.4	1729.1

The results in Tables 1 and 2 show that the nearest-neighbor heuristic is outperformed by the TS approach, at the expense of more computational effort. However, considering the FT approach, the extra time is still very suitable for practical applications. For the LT implementation, CPU times are much greater and possibly of no practical use for large problems. We choose to maintain the LT column to show the huge extra effort necessary to improve on the FT solution.

With respect to the quality of the solutions yielded by the proposed heuristic, we believe that the setup time reduction achieved is quite impressive in economic terms. Comparing the solutions obtained by FHK's heuristic and our approach, it is worth observing that a reduction, say, from 33% to 4% in the makespan reflects great savings on the machine idle times given that the sum of the processing times is a constant.

5. Conclusions

In this article we propose a new heuristic procedure to deal with the problem of scheduling n jobs to m identical parallel machines with sequence-dependent setup times ($P|sds|C_{\max}$). The heuristic is composed of three phases and applies simple

implementations of the Tabu Search (TS) meta-heuristic to escape from local minima. The effectiveness of the method is based on successful removal and insertions' procedures originally developed for solving the Traveling Salesman Problem (TSP). The proposed method was compared to a TSP-based heuristic that makes use of the classic nearest-neighbor procedure. Both of them were compared with an exact method that uses a dichotomous search to achieve optimal solutions. Two TS implementations were tested, Fast Tabu and Long Tabu. Tests were performed on a set of sample problems with 20, 30, 40 and 50 jobs, and 2, 3, 4 and 5 processors. For each combination of jobs and processors ten different instances were solved. Average makespan values achieved by Fast Tabu implementation are 4.68% greater than the optimal solutions, while the error observed in the Frederickson et al. heuristic is 26% in the average.

The running times for Fast Tabu are quite reasonable for practical applications and they increase polynomially with the number of jobs. The Long Tabu implementation yielded greater improvement on Fast Tabu solutions but at the expense of long computational times.

Acknowledgements

The work of Paulo M. França was supported in part by Fundação de Amparo à Pesquisa do Estado de São Paulo (FAPESP), and by Conselho Nacional de Desenvolvimento Científico e Tecnológico (CNPq), Brazil. Michel Gendreau and Gilbert Laporte were funded by the Canadian Natural Science and Engineering Research Council under grants OGP0038816 and OGP0039682, and by the Quebec government under grant FCAR 93ER0289. Felipe M. Muller was partially supported by Coordenação de Aperfeiçoamento do Pessoal de Ensino Superior (CAPES), Brazil. The authors appreciate the many helpful suggestions of the editor and anonymous referees.

References

- [1] Woodruff, D.L. and Spearman, M.L., 1992. Sequencing and batching for two classes of job with deadlines and setup times. *Prod. Oper. Mgmt.*, 1(1): 87–102.
- [2] Serafini, P. and Speranza, M.G., 1992. Production scheduling problems in a textile industry. *European J. Oper. Res.*, 58: 173–190.
- [3] Dobson, G., 1992. The cyclic lot scheduling problem with sequence-dependent setups. *Oper. Res.*, 40(4): 736–749.
- [4] Davis, J.S. and Kanet, J.J., 1993. Single machine scheduling with early and tardy completion costs. *Naval Res. Log. Quart.*, 40: 86–101.
- [5] Ow, P.S. and Morton, T.E., 1989. The single machine early/tardy problem. *Mgmt. Sci.*, 35: 177–191.
- [6] Baker, K.R., 1974. *Introduction to Sequencing and Scheduling*. Wiley, New York.
- [7] Mason, A.J. and Anderson, E.J., 1991. Minimizing flow time on a single machine with job classes and setup time. *Naval Res. Log. Quart.*, 38: 333–350.
- [8] Dearing, P.M. and Henderson, R.A., 1984. Assigning looms in a textile weaving operation with changeover limitations. *Prod. Inv. Mgmt.*, 25: 23–31.
- [9] Sumichrast, R. and Baker, J.R., 1987. Scheduling parallel processors: An integer linear programming based heuristic for minimizing setup time. *Int. J. Prod. Res.*, 25(5): 761–771.
- [10] França, P.M., Gendreau, M., Laporte, G. and Müller, F.M., 1994. A composite heuristic for the identical parallel machine scheduling problem with minimum makespan objective. *Comput. Oper. Res.*, 21: 205–210.
- [11] Graham, R.L., Lawler, E.L., Lenstra, J.K. and Rinnooy Kan, A.H.G., 1979. Optimization and approximation in deterministic sequencing and scheduling: A survey. *Ann. Discrete. Math.*, 5: 287–326.
- [12] Frederickson, G., Hecht, M.S. and Kim, C.E., 1978. Approximation algorithm for some routing problems. *SIAM J. Comput.*, 7: 178–193.
- [13] Parker, R.G., Deane, R. H. and Holmes, R.A., 1977. On the use of a vehicle routing algorithm for the parallel processor problems with sequence dependent changeover costs. *AIIE Trans.*, 9(12): 155–160.
- [14] Clarke, G. and Wright, J., 1964. Scheduling vehicles from a central depot to a number of delivery points. *Oper. Res.*, 12: 568–581.
- [15] Guinet, A., 1991. Textile production systems: A succession of non-identical parallel processor shops. *J. Oper. Res. Soc.*, 42: 655–671.
- [16] Gendreau, M., Hertz, A. and Laporte, G., 1992. New insertion and post-optimization procedures for the traveling salesman problem. *Oper. Res.*, 40: 1086–1094.
- [17] Glover, F., 1990. Tabu search, Part I. *ORSA J. Comput.*, 1: 190–219.
- [18] Glover, F., 1991. Tabu Search, Part II. *ORSA J. Comput.*, 2: 4–32.
- [19] Gendreau, M., Hertz, A. and Laporte, G., 1994. A tabu search heuristic for the vehicle routing problem. *Mgmt. Sci.*, 40: 1276–1290.
- [20] Laporte, G., Nobert, Y. and Taillefer, S., 1987. A branch-and-bound algorithm for the asymmetrical distance-constrained vehicle routing problem. *Math. Modelling*, 9(12): 857–868.